

# Matchmaking: Running Monte-Carlo Simulations to Choose a Suitable Rating System and Matchmaking Policy in One Versus One Games

A Dissertation for the Master of Science  
in Games Engineering at the University of  
Warwick (Warwick Manufacturing Group)

By Gabriel Faes

Supervised by Kurt Debattista

## CONTENTS

|                                              |    |
|----------------------------------------------|----|
| Abstract.....                                | 3  |
| I. Introduction .....                        | 4  |
| Matchmaking: .....                           | 4  |
| Rating Systems: .....                        | 6  |
| II. Literature Review .....                  | 8  |
| Rating Systems: .....                        | 8  |
| Matchmaking: .....                           | 10 |
| III. Research Outline .....                  | 13 |
| IV. Research Method .....                    | 15 |
| Introduction: .....                          | 15 |
| Simulation Variables:.....                   | 15 |
| Metrics: .....                               | 16 |
| Player-Joining Behaviour:.....               | 17 |
| Matchmaking Policies: .....                  | 18 |
| Final Variables: .....                       | 19 |
| V. Results .....                             | 21 |
| Rating Error: .....                          | 21 |
| Non-Continuous Match Skill Disparity: .....  | 23 |
| Continuous Matchmaking:.....                 | 24 |
| An Example with <i>TETR.IO</i> :.....        | 27 |
| VI. Conclusion .....                         | 29 |
| References.....                              | 31 |
| Appendix: Additional Experiment Results..... | 33 |
| Elo vs. Win Count: .....                     | 33 |
| Bradley-Terry Bias for OpenSkill: .....      | 33 |
| <i>TETR.IO</i> Example: .....                | 34 |

## ABSTRACT

With the rise of online competitive games, matchmaking has become increasingly important to both players, wanting balanced matches quickly, and developers, where player retention can affect the revenue of the game. Recent research has focused on developing new rating systems or matchmaking policies, but how one affects the other is relatively unknown. Knowing the relationship between the two could inform how applicable research on matchmaking policies is when applied in broader contexts than their original specific use cases, such as with a different game and/or rating system. Focusing on one-versus-one games and Skill-Based Matchmaking, this paper investigates, using Monte-Carlo simulations, the relationship between rating systems and matchmaking policies. The research aim is to find whether they can be designed largely independently or whether their implementation requires careful consideration because the choice of a rating system could affect the performance of a matchmaking policy when compared to other matchmaking policies. The simulation results strongly support the implementation independence between rating systems and matchmaking policies, as the choice of the rating system does not meaningfully affect the performance of matchmaking policies in relation to each other. These simulations are also used to check the performance of more recent and open-licence rating systems, namely Glicko, Glicko-2 and OpenSkill, comparing them to each other and Elo. Using the simulations, matchmaking policy suggestions can be made, without considering the specific context of a game. The code is also available to run simulations with different variables for developers to use and find better matchmaking policies for their specific use case.

# I. INTRODUCTION

## MATCHMAKING:

Matchmaking in online competitive video games is ubiquitous nowadays. As competitive games grew in popularity over the last few decades, so did their need for fun and balanced matches. Games can now individually reach tens of millions of monthly users, with some having more than half a million at any one time (Statista Research Department, 2025) (Valve, 2026). These can each generate billions in revenue, and so it has become increasingly important to ensure the matchmaking system of the game as a whole works to the best of its ability.

While team games are now the most popular online competitive games, two-player competitive games remain prevalent, notably in online board games, fighting games, or real-time strategy games, with individual games still attracting millions of monthly users. One-versus-one matchmaking also forms the basis of matchmaking in team games and is, therefore, a useful starting point to study matchmaking more generally.

Older games used a lobby or list system, where a player would host the game (or lobby), and others would explicitly choose which lobby to join. Nowadays, a matchmaking system is used. It pairs players waiting in the “matchmaking queue” together (without the players being able to choose who they are paired with, in contrast to the lobby system) in an attempt to give them fun and fair matches quickly. As an example, Lichess (an online chess website) uses both matchmaking queues, for quick matches, and the lobby system, for different chess rulesets (Figure 1 shows Lichess’ matchmaking screen). The set of rules used by a game’s matchmaking to pair players is called the matchmaking policy. Different matchmaking policies can give drastically different pairings, or the same pairings but make the player wait different amounts of time. Which exact matchmaking policy to choose depends on the game and the objective of the matchmaking system, but those basic objectives are fairly similar across competitive games.

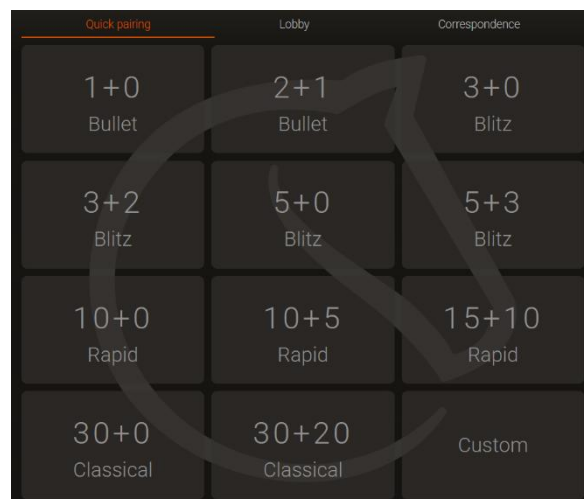


Figure 1: The Lichess “quick pairing” matchmaking screen, where players can be matched against others to play chess online. Each time control is a different matchmaking queue. Lichess also uses the lobby system on a separate menu if players wish to play different game modes or time controls.

The general purpose of a good matchmaking system is to make the game more fun and engaging. This can both increase the overall enjoyment of the player (making it more likely that they recommend the game to a friend for example) and keep them playing. Engagement has become a focus of matchmaking systems recently. Games benefit from having more players in lots of ways, especially cooperative or competitive games, as having an ample number of players keeps queue times low and gives players more balanced and fun matches. The game is more likely to attract new players and keep them engaged, who can then themselves attract new players.

The matchmaking process is fairly similar from the player’s point of view regardless of how players are paired behind the scene, or even which game they are playing. The player enters the game’s (or a specific mode’s) matchmaking queue. They then wait a certain amount of time until the game finds their opponent(s) (and potential teammates), at which point the game starts. Given this, the basic

objective is to assign the best match possible to the player without letting them wait too long. What determines “the best match possible” depends on the needs of the developers and the specific game, although the basic objective is generally for matches to be balanced, with each player having an equal chance of winning (achieving a 50%-win rate in one-versus-one games). This balanced target is called Skill-Based Matchmaking (SBMM). SBMM policies may still sometimes give deliberately more unbalanced matches to curb waiting time. Choosing how to tune the specific variables in a policy can change which factor a matchmaking policy will consider more important (such as prioritizing low wait times, or balanced matches).

SBMM also informs other more complex matchmaking objectives, such as Engagement-Oriented Matchmaking (EOMM) which attempts to keep the player playing for as long as possible and ensure they come back to play (by giving them harder matches after easier ones for example, or vice-versa). The exact objectives of SBMM and EOMM are slightly different (the former prioritizing balanced matches, and the latter player engagement), but both use the same player characteristics to pair players. EOMM is more complex and generally game-specific. Different games have different engagement objectives, and how or when a player decides to stop playing is incredibly difficult or impossible to determine in a general way without studying a specific game. This makes it much harder to study EOMM in a general way and, therefore, SBMM will be the matchmaking target for this study.

If all players were available, the problem would be akin to a Swiss tournament<sup>1</sup>. The matchmaking policy used by Swiss tournaments each round is to simply pair players by the number of wins they have (or more precisely the match record they have). This is one of the simplest matchmaking policies and works well as long as all players are available for all rounds. Unfortunately, player availability in competitive online games is not universal, and generally only a small fraction of the player population is available at any one time. Players can also decide to join (or leave) the matchmaking queue whenever they want, instead of at fixed rounds or intervals. This partial and dynamic player availability is one of the major issues an online competitive game’s matchmaking must deal with.



Figure 2: The matchmaking screen for the ranked “Tetra League” mode of TETR.IO, a falling block puzzle game akin to Tetris. It features a single matchmaking queue. This competitive mode uses Glicko-2 for its rating system.

<sup>1</sup> A 1-vs-1 tournament format with a fixed number of rounds; all players participate in each round. The winner is the player having won the most matches (with draws counting as half a win).

## RATING SYSTEMS:

In order to give balanced matches, matchmaking policies need to know or estimate a player's skill. To do so, they use a rating system. The objective of rating systems is to predict the likelihood of a player winning against another, they do so by estimating a player's skill, which is represented by one or more parameters. This is incredibly useful for matchmaking as it allows the matchmaking policy to make better decisions as to who to pair together. Rating systems are used extensively outside games to rank products based on customer feedback, and in games or sports they also have other uses besides matchmaking, mostly for sports betting and spectator entertainment, but also as seeding (initial placement) for tournaments (which could also be viewed as a kind of matchmaking).

Early methods of evaluating players' skills for games were incredibly simple, such as counting wins (like in a Swiss tournament), or the difference between wins and losses. Some slightly more sophisticated systems are still in use today, notably in the ATP (Association of Tennis Professionals) Tour, where tournament winners are given points depending on the tournament's level (or how important the tournament is) (World Tennis, 2019). These systems actually perform reasonably well for players participating in many matches, but can be easily skewed or biased, and do rather poorly at predicting the winner between two players who participate less often.

As games grew in popularity during the 20<sup>th</sup> century, more formal rating systems were developed. Elo, primarily developed for chess, was the first popular and successful rating system; it is still widely in use today. More research into rating systems has been done more recently thanks in part to the surge in online competitive games and more rating systems have been developed, a few of which have seen widespread use, most notably Glicko and TrueSkill (Figure 2 shows the ranked mode for *TETR.IO*, which uses Glicko-2 as its rating system, an improvement over the original Glicko rating system).

These modern rating systems give the player a number (their rating), and update that number after every match the player plays (Figure 3 shows a simplified diagram on how rating systems update each player's rating). How efficient a rating system is at estimating a player's skill can greatly affect how good a game's matchmaking is. A rating system which manages to more effectively estimate a player's skill and how they will fare competitively against others can give more balanced matches, and can do so earlier for newer players who have played fewer matches.

The choice of which rating system is best is dependent on context, and the rating systems can often be optimized for the specific game in question (e.g., some games may involve more luck than others; knowing by how much could optimize the rating system for that game). Still, determining more efficient rating systems compared to others in a general context can be useful for developers who wish to use these "pre-configured" rating systems rather than custom-made ones for their game. Most studies compare a new or their new rating system with Elo, often finding favourable results, but few compare them directly to each other.

More research has been dedicated to finding optimal matchmaking policies, but they mostly apply to individual games or specific scenarios. Existing research evaluates matchmaking policies conditional on a rating system, but it is unclear whether conclusions about the relative performance of those policies may depend on the choice of a rating system.

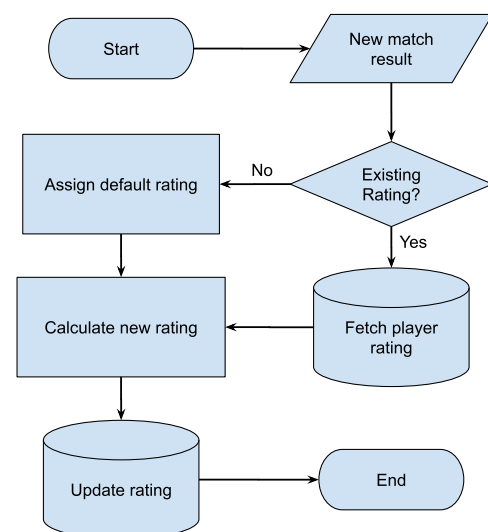


Figure 3: A process flowchart for how most rating systems update a player's rating after every match.

This informs the main research question: whether the choice of rating system affects which matchmaking policy performs better. Research into this specific problem is scant, and can be useful to inform the design of matchmaking policies and rating systems, especially when it comes to their interactions with each other. If the rating system choice does not change how a matchmaking policy performs compared to other policies, it means developers can independently create their rating system (making it the best match predictor possible) and their matchmaking policy (optimizing it to find the best pairings possible) without needing to check whether the rating system may affect the policy's reliability. If some matchmaking policies perform better than others for a specific rating system, knowing how the choice of rating system affects the matchmaking policies would be beneficial to inform future research and inform developers on how to develop more efficient matchmaking.

## II. LITERATURE REVIEW

In order to properly investigate the correlation between rating systems and matchmaking policies, it is useful to study them independently first. Rating systems and matchmaking are relatively specialised areas of research, and there remains considerable scope for further study within these fields. A substantial proportion of the research has been performed in the 21<sup>st</sup> century, with an acceleration in the last decade, particularly in the area of matchmaking. However, the first successful rating system goes back to Elo in the mid-20<sup>th</sup> century. The Elo rating system laid the foundations for much of the subsequent research in rating systems, making it a natural starting point for this review.

### RATING SYSTEMS:

Elo was one of the first rating systems created. Developed by Arpad Elo at the end of the 1950s, its main use was to quantify the skill of chess players (Elo, 1961). It was adopted by the International Chess Federation (FIDE) in 1970 after having been in use for ten years in the US Chess Federation (USCF) (FIDE, 2025) (Chess.com, n.d.). Other rating systems were developed before then (such as the Harkness system); these mostly considered the win rate of a player against other players, either over a few matches (such as a tournament), or longer periods of time, but frequently yielded inaccurate ratings and could easily be exploited by players (Glickman M. E., n.d.). Nowadays, numerous games use Elo as a basis for their own rating systems; few to none use it without some modification however, the simplest of which, in use by FIDE, is a dynamic  $k$ -factor choice.

In the Elo system, given a player's rating  $r$ , the opponent's rating  $r^*$  and a new result  $s$ , the player's new rating is

$$r' = r + k(s - p)$$

where

$$p = \frac{1}{1 + 10^{\frac{r^* - r}{400}}}$$

and  $k$  a factor generally set to 32 (the factor  $k$  in modern implementations is sometimes set higher for new or lower-rated players and lower for higher-rated players). In other words, a player's rating change is equal to the product of  $k$  and the difference between the observed result and the expected result ( $s - p$  being this discrepancy, with  $p$  being the probability of the player winning against the opponent). This forms the basis of the Elo system and many other systems that followed it.

The Elo system, while elegantly simple, is particularly slow at converging. The number of matches required for an accurate estimate of a player's skill can be very high. This is because the maximum change each match is capped by  $k$ . Arpad Elo was aware of this drawback and implemented an initial rating estimation for the player based on the player's win rate and average opponent skill in their first few matches. This, however, still requires a certain number of games before any skill estimation can be done (Elo recommended at least 6 matches) (Elo, 1961). More modern implementations tend to forego estimating a new player's rating in favour of immediately giving that player an initial rating (for example, 1400) and raising  $k$  significantly for their first few games (even as high as in the hundreds). Still, this approach can result in a player's rating exhibiting substantial rating oscillation before converging.

To fix this analytically, rather than guessing what  $k$  should be set to, Glickman proposed a new Bayesian rating system called Glicko. A player now has both a rating  $r$  and a rating deviation  $RD$ , with starting values fixed at 1500 and 350 respectively. A player's "true" skill lies inside of the normal distribution

with mean  $r$  and standard deviation  $RD$ . Therefore, a new player's rating is expected to eventually lie between 800 and 2200 with 95.45% certainty. At its core, Glicko is similar to Elo. The rating deviation  $RD$  is similar to Elo's  $k$  factor, but it now is a parameter of an individual player's rating. This rating deviation is updated after each game, almost always diminishing (as Glicko becomes more confident of what the player's rating is), but can also increase with player inactivity (Glickman M. E., The Glicko system) (Glickman M. E., Parameter estimation in large dynamic paired comparison experiments, 1999).

Glicko-2 is an upgrade made to the Glicko rating system by Glickman which introduces a new variable called "volatility". Volatility measures the uncertainty of the player's overall results, and is iteratively updated along with the regular rating and rating deviation updates (Glickman M. E., Example of the Glicko-2 system, 2001) (Glickman M. E., Dynamic paired comparison models with stochastic variances, 2001). It then influences the rating deviation change, with more volatile results slowing down the rating deviation decrease (or even increasing the rating deviation in some cases). While it is not mathematically used as such in Glicko-2, viewing volatility as the standard deviation to the rating deviation is a good analogue for how volatility works.

While Elo ratings are easy to compute by hand or with a simple calculator, Glicko (or Glicko-2) ratings are not and require a computer to calculate any meaningful number of player ratings. Its complexity is what leads a lot of games to simply choose the Elo system (with small modifications) rather than implement Glicko. Furthermore, because of its simplicity, modifying Elo slightly to fit a particular game's needs is much easier than with Glicko. A common use case for this is modifying the rating system for games involving more than two players.

TrueSkill™ is a rating system created and trademarked by Microsoft Research which provides a built-in way to rank players in team or free-for-all (FFA) games. The algorithm uses a factor graph to update all players' ratings based on the match result. This graph and subsequent rating updates get increasingly complicated as the number of teams and players increase, nevertheless, the update algorithm can be optimized to be only a small multiple of the run time for Elo (Herbrich, Minka, & Graepel, 2007). The TrueSkill team achieved a significantly better prediction result when comparing to Elo in their paper, but made no comparison to the Glicko system.

Because of the trademarked nature of TrueSkill, another rating system, OpenSkill, was created as an open license alternative to TrueSkill. It reports faster calculation times and comparable results to TrueSkill, and is free to use. Its underlying algorithm is heavily based on Weng and Lin's work, which is another Bayesian rating similar to Glicko, and states competitiveness with TrueSkill (Weng & Lin, 2011).

Other rating systems exist; some are presented here to provide context and a broader overview of the field, rather than as methods considered in this study. QuickSkill uses machine learning to quickly estimate the skill of new players. It uses in-game and game specific knowledge to estimate their rating (based on previous players' first few games and the rating they ended up having later) (Zhang, et al., 2022). While being able to accurately estimate a player's skill after only one or two games is incredibly useful, its use of in-game knowledge can make it biased if players are aware or figure out which factors are considered by the rating algorithm (objectives which do not strictly serve to win the game can generally be exploited by manipulative players). Because this rating system requires game specific knowledge, it needs to be trained on data coming from one specific games, and cannot be used in a wider context.

Elo-MMR is a multiplayer extension of Elo based on the Glicko rating system. They show that Elo-MMR is "incentive-compatible", which means a player who wishes to maximise their rating will never

deliberately underperform (Ebtekar & Liu, 2021). This is also fulfilled by Elo naturally (unless the  $k$  factor is not constant), but is harder to prove for Bayesian rating systems (such as Glicko, TrueSkill, and OpenSkill) depending on how their rating standard deviation is updated (for example, in Glicko-2, significantly unexpected results can cause the volatility to increase which can, in time, cause the standard deviation to increase; this can incentivize a player to deliberately have unexpected results). Elo-MMR's stated improvement is in free-for-all games (with comparable results to TrueSkill) and it is still seldom used.

This leads to the four rating systems studied: Elo, Glicko, Glicko-2, and OpenSkill. These are all open-license and are extensively used, or are a stated alternative to an extensively used trademarked rating system (in this case OpenSkill for TrueSkill).

#### MATCHMAKING:

While the basics of matchmaking have existed for a long time, such as in auctions or sports (or even early chess tournaments), online competitive games have asserted the need for better matchmaking policies to both cope with the number of players and give them entertaining matches. These games must also deal with the player's partial and dynamic availabilities, which are problems which are generally not present in other forms of matchmaking. Players expect matches which are fair and for those matches to be found quickly. This forms the basis of the loss function most matchmaking policies are compared against: the time spent searching for a match, and how balanced the match is once found. How these two factors are weighted relative to each other is heavily dependent on the game and the developer's objectives; some games may find it more appropriate to wait longer for a better match, while others (perhaps with shorter match times) may take a less optimal pairing earlier.

The reason players want balanced matches is because these are more fun. The fun of a game generally comes down to its difficulty. This can be explained by the flow theory: if the game is too easy, the player will feel bored, while if it is too hard, they will feel frustrated. Getting that difficulty perfectly right may have the player enter a "flow state" where they are wholly focused on the task at hand and forget about the environment around them. Achieving that perfect balance for a chance at the player entering that flow state maximises enjoyment (Csikszentmihalyi, 1990) (Biasutti, 2011).

In balanced one-versus-one competitive games, the game is designed to be won 50% of the time by each player (in contrast to asymmetrical games, where one player may have a deliberate advantage). This leads to a target of 50% for the win rate of each player when attempting to find a match. However, if it were possible, a slightly higher win rate may be beneficial. Players dislike losing more than they like winning, due to loss aversion (Blavatsky, 2024). Unfortunately, a higher win rate is impossible in most games due to them being zero-sum; increasing one player's win rate would decrease another player's win rate. Games with different designs which are not zero-sum may find it beneficial to aim for different matchmaking targets.

A 50%-win rate is the general target for matchmaking, and is called Skill-Based Matchmaking (SBMM), but some games have started going further to maximise engagement and keep players playing for as long as possible; this is called Engagement-Optimised Matchmaking (EOMM). Most of it is designed to reduce "churn", which is when a player leaves the game, or worse, does not come back in the next week or month. Some specific patterns are identified that result in a higher probability of a player churning (such as a specific pattern of wins or losses), and players are then specifically matched to reduce those patterns as much as possible. Whether using SBMM or EOMM, games need to decide how early they wish to take a non-optimal pairing compared to waiting longer for a chance at a better match.

While the earliest online games used a lobby or list system, competitive games now use a matchmaking queue. Players wait in the queue and get matched with other players based on several factors, a non-exhaustive list of which includes skill (calculated using the rating systems above), ping (the internet connection delay between players), time spent in the queue so far, but also sometimes the past match results of the players, and even their account standing.

Developers use these factors to determine the matchmaking policy which best fits their game and objectives. For example, prioritising low wait times over any other factor would result in players getting paired together as soon as there are two players in the queue. *Tetris Effect: Connected* uses such a matchmaking policy with one additional restriction: players must be within a certain rating range of each other; therefore, if one player is waiting and another player joins the matchmaking queue and both players are within acceptable range, they are matched with each other, otherwise they both wait until another player within their range joins. Such a policy is simple to implement and may be appropriate for games with a low number of players, but can easily cause players to either wait a very long time (if the rating range is too small) or can generate wildly imbalanced matches (if the rating range is too large).

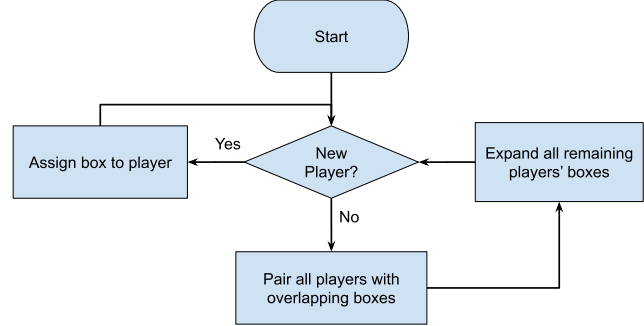


Figure 4: A simplified flowchart that represents how some matchmaking policies work. The specifics of the three processes in the flowchart determine the matchmaking policy; changing how any of those processes works would change the matchmaking policy.

These “simpler” policies can be represented in a multi-dimensional space  $\prod_{i=1}^n X_i$  where  $X_i$  may be any set, most commonly the real numbers; each dimension represents a different factor the game wishes to consider (such as latency or rating). When a player joins the matchmaking queue, they are assigned a box  $f: t \rightarrow \wp(\prod_{i=1}^n X_i)$  in that space; the longer the player waits, the more the box may expand in some of the dimensions. Then, either once two players’ boxes overlap or (depending on the matchmaking policy) when both player’s boxes overlap significantly (such as at least half in some or all dimensions), then they are matched together (Figure 4 shows a flowchart for how these kinds of matchmaking policies work). For *Tetris Effect: Connected* for example, this space would be  $\mathbb{R}$  with the box  $f(t_0) = [r, r]$  and then immediately updated to  $f(t_n) = [r - k_0, r + k_1]$  if no opponent is found. This is a segment (where  $r$  is the player’s rating and  $k_0$  and  $k_1$  are real numbers set by the game). Because the segment does not continuously expand as the player waits, players must wait, perhaps indefinitely, until an appropriate player joins the queue, as they can never be matched with players also waiting in the queue.

More recent and experimental matchmaking policies consider the potential losses generated by each pairing, and minimize those losses dynamically (either through a specifically tuned algorithm, or using machine learning). This can be represented as a graph, with each player a node, and each vertex between nodes having a value for the error loss that match represents, as seen in Figure 5. This has been successfully used to

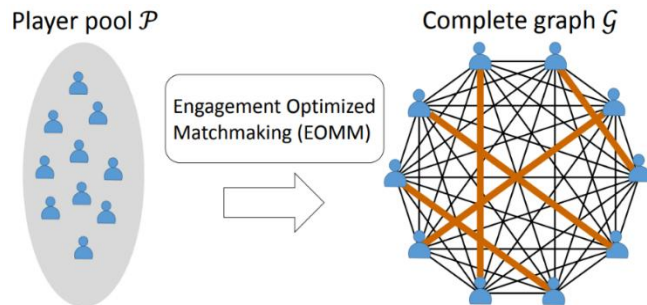


Figure 5: An EOMM matchmaking policy model. The objective is to find the optimal pair assignments on the graph. Figure taken from (Chen, et al., 2017).

generate EOMM policies to maximise engagement in “a popular game from Electronic Arts” (Chen, et al., 2017). Performing simulations on data from that game, their method results in a statistically significant increase in the retention of players.

Because of the nature of matchmaking, testing matchmaking policies with their corresponding rating system directly against real data is difficult. Knowing the winner of a match between pairings which have not occurred between real players is impossible, and real competitive video game data is naturally biased by the matchmaking policy (and rating system) chosen for that game: the data on when players join the matchmaking queue (and leave), how many matches they play, how long their matches are, and more, are all influenced by the game itself and how the matchmaking policy pairs players (how quickly and with whom). Comparing rating systems against real data to determine the best one is much simpler (albeit still marginally biased), but comparing matchmaking policies requires simulations to get meaningful results.

As of writing, there are no papers which simulate different combinations of matchmaking policies and rating systems. Most simulations are run against a single matchmaking policy with one rating system, a single policy with multiple rating systems, or multiple policies with a single rating system. Simulations testing different combinations of rating systems and matchmaking policies together to determine how one affects the other to optimize matchmaking queue efficiency are lacking. While one recent theoretical mathematical result (which has not undergone formal peer review yet) suggests that the matchmaking policy and rating system do not affect each other (Nozawa, 2025), no simulations have been run to prove or disprove this.

### III. RESEARCH OUTLINE

By running Monte-Carlo simulations to test different combinations of popular, open-source rating systems and matchmaking policies, the research aim is

- Can the implementation of rating system and matchmaking policy be separated when considering the performance of matchmaking?

Changing the rating system should not affect the performance of a matchmaking policy compared to other matchmaking policies for the independence of rating systems and matchmaking policies to hold. It is important to clarify that the rating system will always affect the performance of a matchmaking policy. What matters is whether a matchmaking policy performs worse than another policy with one rating system, but better than that other policy with another rating system. If this is the case, then the independence does not hold. To measure whether the performance of matchmaking policies is affected, specific metrics (match skill disparity and wait time) are introduced in Chapter IV and subsequently calculated and discussed in Chapter V.

If this independence does not hold, the objective is to determine how the choice of rating system and matchmaking policy affects matchmaking efficiency. Disproving the independence requires only a counter-example in the simulations run. However, it will not be possible to prove the independence with Monte-Carlo simulations as that would require testing every matchmaking policy with every rating system possible. Nonetheless, having evidence for the independence is useful to inform future matchmaking work.

When running these simulations, as discussed in Chapter II, all data is simulated. A matchmaking policy will inevitably make different pairings compared to the ones the matchmaking policy in the real-data made, and results for those matches would not be available. How this data is simulated is discussed in Chapter IV.

Additional research objectives can be considered by running these simulations.

- Determine, from the selected rating systems, whether one performs better at predicting match results, or in which contexts some rating systems perform better than others.

For this, another metric (rating error) is introduced in the next chapter. This research objective is useful for developers who will not implement their own rating system, as they can select the rating system which best suits their needs without having to modify it. It is also useful information for those who will develop their rating system as they can choose a more suitable starting point or develop it more efficiently by knowing how and when some rating systems perform better.

- The simulations may provide matchmaking policy suggestions in a general context (without choosing a specific game).

Just like the first research objective, this can help developers choose a more suitable matchmaking policy without having to run specific tests, and can direct developers who will run simulations to their chosen matchmaking policy more efficiently.

- The simulations may provide a framework to choose a matchmaking policy in a specific context (by running simulations approximating the player behaviour and player population of the chosen game).

This is discussed at the end of chapter V; the code can be used for future research or to aid developers in running their own simulations. This can directly help developers choose the “best” matchmaking

policy for their specific game, but requires running simulations directly using the data for that game (unlike more general matchmaking policy suggestions).

## IV. RESEARCH METHOD

### INTRODUCTION:

Introduced here is the way the simulations will be run to determine whether or not rating systems affect the performance of matchmaking policies when compared to other policies. They will also help find the best rating systems, matchmaking policies, or combinations of both if the independence does not hold.

To do so, Monte-Carlo simulations are performed. Monte-Carlo simulations consist in running several simulations to gather repeated random samplings to calculate some numerical result. It is incredibly useful when mathematical analysis to calculate the exact value would be too computationally expensive, but calculating a close approximation is not. The simulations here will provide a data point and then average the data points to obtain an estimate of the true value of the measured metric. As more simulations are performed, the average value converges towards the true value.

The Monte-Carlo simulations will simulate a matchmaking queue with players, matching them using different combinations of matchmaking policies and rating systems, as well as different “player-joining behaviours” (how players join the matchmaking queue). For each combination, the error of some specific metrics can be calculated and compared.

The code uses Python 3.11 and is available at <https://github.com/Gabrio05/RatingSystemComparer>.

### SIMULATION VARIABLES:

Four rating algorithms are implemented: Elo, Glicko, Glicko-2, and OpenSkill. Elo because it is the most widely used rating system and is simple to implement in games. Glicko because it is now used by numerous games and enjoys a higher stated accuracy. Glicko-2 as a comparison to Glicko to test its improvement. And OpenSkill for its stated competitiveness with TrueSkill and as a more modern comparison point. All four are public domain and can be used for any purposes, fuelling the need for objective comparisons.

The Bradley-Terry model is used for these simulations. It is a widely used pairwise probability comparator. Given two player skills  $p_1$  and  $p_2$ , the probability of player 1 winning is given by

$$\frac{p_1}{p_1 + p_2}$$

with the complement being the probability of player 2 winning. While this model does enforce a strict ranking among players because it forces skill transitivity (player skills are fixed regardless of their opponent), and this property may be violated in practice on small segments of the player population (generally between players relatively close in skill), given a large enough population and skill distribution, those differences are small enough for the Bradley-Terry model to be appropriate. It is also, notably, simple and non-arbitrary. More complicated models, such as hierarchical models, are significantly more difficult to model real player populations against.

Each simulated player is given a skill from a probability distribution on  $\mathbb{R}_+^*$ . The skills generated should not be linear but rather involve orders of magnitude of differences in skill, else the top of the population pool is too close in skill to each other (the top player, in a linear distribution for example, would only win two thirds of their matches against the average player, which, while might be appropriate for some games, is generally not the case). The one used in the simulations generates a number in the normal distribution (using a mean of 0 and standard deviation of 1.2) and then makes it the exponent to the base 10 (i.e.,  $10^{\mathcal{N}(0,1.2)}$ ).

This, as seen in Figure 6, generates a certain skill distribution which seems to closely mirrors those seen in games (most of the players are concentrated in the center, with more skill outliers on both sides of the curve). This includes Chess, with both the FIDE rating distributions<sup>2</sup> (FIDE, 2026) and Lichess (the biggest free chess website) rating distributions regardless of mode (Lichess, 2026). The standard deviation of 1.2 was chosen arbitrarily, with similar values leading to the same comparisons and overall results (a higher standard deviation generally leads to higher errors, while a lower one lowers the errors); significantly higher or lower standard deviations were not considered as they led to player populations that did not closely resemble those seen in games.

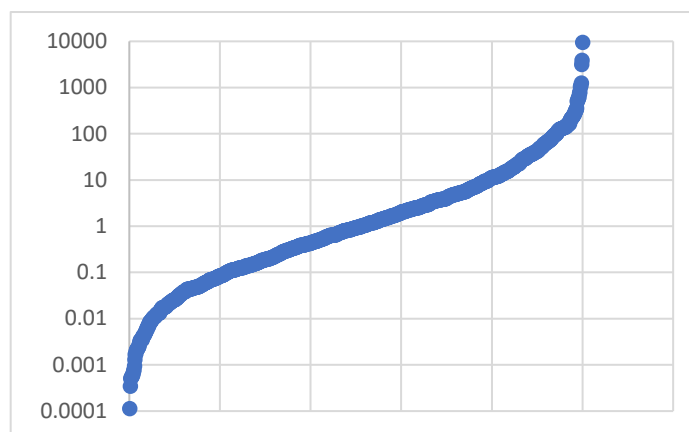


Figure 6: 1000 randomly generated skills from the distribution used in simulations

Each simulation has 1000 players. This is because the chosen matchmaking algorithm and rating system are not substantially affected by adding more players past one thousand. The variance between simulations at one thousand players is already low enough that it is more practical to run more simulations than bigger ones.

When a match has been generated by the matchmaking policy, the Bradley-Terry model gives the probability of one of the players winning. A random number in  $[0, 1)$  is generated and that player wins if the number is lower than the probability. The total number of matches generated must be similar for the errors to be comparable between simulations. The matchmaking policies may only pair players together based on the match history of a player; in these simulations, this is the player's rating calculated by the rating system. Games often use other factors such as the player's latency to the server, or even their behaviour (for instance pairing together bad actors or players who tend to leave matches before they are finished). These have not been taken into account as they would not change the errors calculated, but only lengthen the simulation time and increase the variance between simulations.

For each experiment, 15 simulations are run. This is an appropriate number as the final results did not change significantly with more simulations; the variance of the Monte-Carlo results was low enough.

#### METRICS:

Rating systems attempt to estimate a player's true skill by computing their rating. To calculate the discrepancy a rating system has with the true skill of players, the rating error is calculated. At the end of each simulation, the rating system produces a probability of victory for one player in each pair of players (because of the symmetric nature of one-versus-one matches, calculating this probability on only one player is sufficient). The error is defined as the difference between the probability estimated by the rating system and the probability given by the Bradley-Terry model. The mean squared error (MSE) is then computed over all player pairs. While this metric is not what determines the performance of a game's matchmaking, it is a practical way to directly compare the accuracy of each rating system and can inform how well each rating system may perform in the following metrics, as well as answer the first research objective.

<sup>2</sup> FIDE employs a rating floor, which groups the bottom players, and what would normally be the lower tail of the rating distribution, together. However, the upper tail of the normal distribution is still visible.

Other ways of calculating the rating error may be appropriate when using real-world data, such as the Brier score or log loss which would compare the probability prediction against the actual outcome of the match. However, because the true exact probability is known (given by the Bradley-Terry model) and these rating systems have built-in methods to estimate the probability of one player winning, calculating the error directly over those probabilities is more informative. Other non-probabilistic models exist, such as Spearman's rank correlation coefficient or Normalized Discounted Cumulative Gain (NDCG), which use the players' ranking given by the rating system to compare against the true ranking of the players (Evidently AI Team, 2025) (Volk-Jesussek, 2026). However, these metrics focus on the player ranking, and punish every incorrect order the same, whether the rating difference between those players is large or small.

The metric which determines the matchmaking performance is the match skill disparity error. For each match, using the probability given by the Bradley-Terry model, the error is the difference between that probability and a theoretical perfectly balanced match with a 50% chance of either player winning. The MSE is computed over all matches to obtain the match skill disparity error. This is a very useful metric as it incentivizes matchmaking policies to always give closely balanced matches; squaring the error also avoids the policy from giving one incredibly balanced match for one completely unbalanced. This metric focuses on SBMM rather than EOMM as the latter is highly game-dependent, currently seldom used in one-versus-one games, and still relies on SBMM as its default objective.

The last metric is used for continuous matchmaking, which is one of the player-joining behaviours explained below. It is the wait time of a player to find a match (the number of rounds a player had to wait before being matched). Again, the MSE is calculated over every match, once for each player (a player waiting for hundreds of rounds would not meaningfully affect the wait error without squaring first). Waiting longer will naturally lead to more balanced matches as more players join the queue; one of them is more likely to be closer in skill to one already waiting, so the balance between the match disparity error and the average wait time needs to be carefully considered depending on the game and player population.

#### PLAYER-JOINING BEHAVIOUR:

How players join the queue is important to determine what the optimal matchmaking algorithm could look like. Three main ways of players joining the queue were tested: single-round, multi-round, and continuous. Continuous matchmaking is how real matchmaking queues function, but the other two types are very useful to inform what the matchmaking policies for continuous matchmaking should look like. Single-round refers to generating all the simulation's matches at once (and, therefore, randomly). Because neither the rating system nor matchmaking policy can influence the matches generated, it is a useful way to directly compare the efficacy of the rating systems. Multi-round refers to generating matches in rounds, with a fixed number of rounds. After each round, the matches generated have their players' ratings calculated, and those ratings can influence the next round of matchmaking. If all players are available each round, this is exactly the same as a Swiss tournament. Continuous matchmaking is the same as multi-round, except the matchmaking policy may choose not to pair certain players each round (making them "wait"), and they will automatically be considered for the next round.

How often an individual player joins the next round can vary the results of the simulation by a significant amount (with one of the biggest reasons being the number of matches being played is different). Some multi-round simulations have a selection probability of 1 (meaning every player is available and expecting a match) and some with a selection probability of 0.5 to compare. Continuous matchmaking is more complicated as many more rounds are performed.

A selection probability of 0.01 for continuous matchmaking was considered at first (therefore adding around 10 players per round). However, it is too easy for the matchmaking policy to find suitable pairings with that many players available. Because of this, the continuous matchmaking simulations were run with a selection probability of 0.001. This means around one player joins the queue every round. While having exactly one player join each round was considered (essentially simulating a game where the matchmaking policy is run only when a player joins), this cannot emulate long periods of waiting, where no players join for several rounds (which can be fairly common).

Regardless of how often a player joins a round, the number of matches for all simulations is fixed at 5,000, and 50,000, and the number of matchmaking rounds played adjusted so that it closely matches those values (this is particularly relevant for continuous matchmaking, as often hundreds of thousands of rounds are run). These correspond to 10 and 100 matches per player on average respectively.

#### MATCHMAKING POLICIES:

Different matchmaking policies are tested, which also differ depending on the player-joining behaviour. For single-round matchmaking, only a random matchmaking policy is tested (the rating system cannot affect the matches being generated). Multi-round matchmaking has three different policies tested, along with a random matchmaking policy as control.

These matchmaking policies first sort all players available for the round by their rating, and then, for each player without a match starting from the highest rated player, chooses their opponent according to different rules. The “Stairs” matchmaking policy simply chooses the 2<sup>nd</sup> highest player as their opponent (resulting in every player being paired with their neighbour). The “Batched” matchmaking policy randomly chooses one of the next 50 players to be their opponent. Finally, the “Skill Weighted

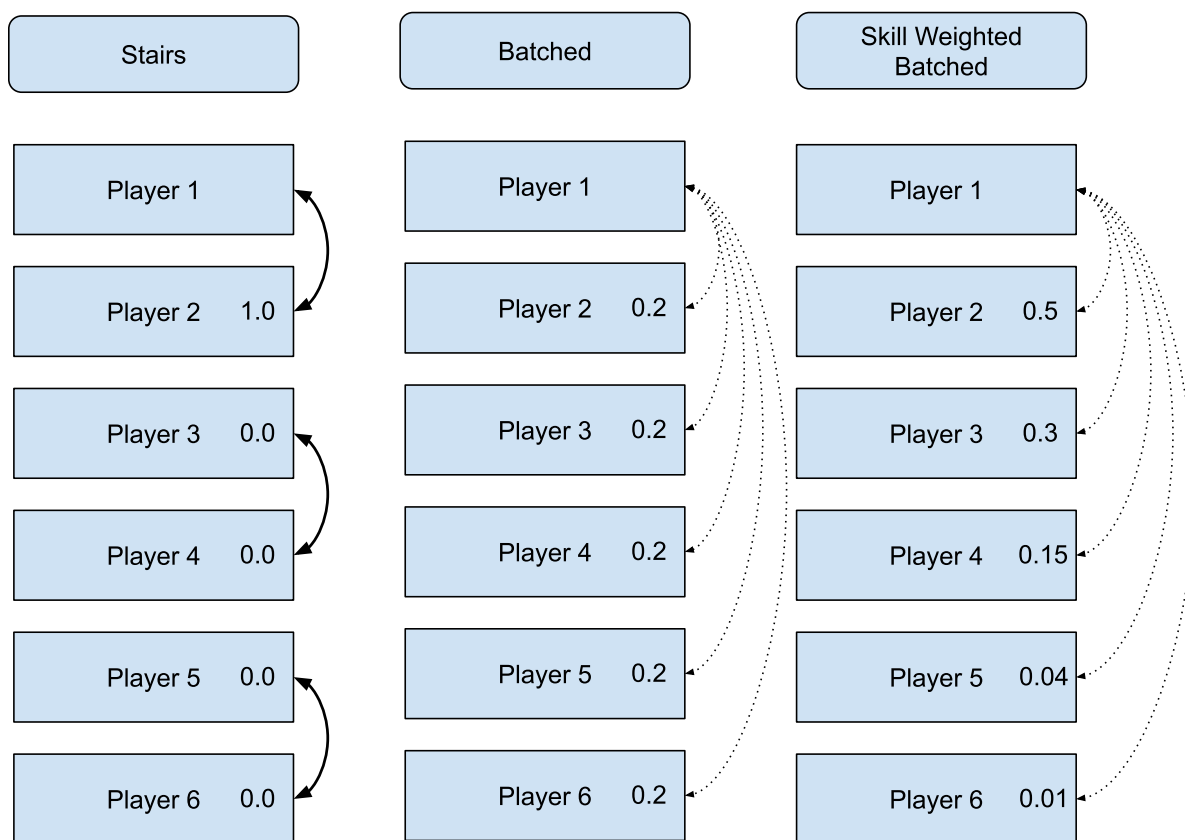


Figure 7: A diagram representing how the three multi-round matchmaking policies work. Solid arrows are guaranteed pairings. The number next to each player represents the probability of that player being paired with Player 1 (these are approximate for Skill Weighted Batched). Both Batched policies consider 50 players in the diagram instead of the 50 used in the simulations.

Batched” matchmaking policy chooses one of the next 50 players randomly, but with the probability weighted by the difference in rating with that player. Figure 7 shows how the three matchmaking policies work.

For continuous matchmaking, the matchmaking policies tested were that of the “Box” matchmaking policy with different values for the acceptable pairing range. This class of matchmaking policies is similar to that of “Stairs”, except the policy may choose not to pair a player this round if there are no other players close in skill to them.

There were three main variables tested for this “Box” policy: the initial rating opening, the rating expansion each round, and the range overlap required for the pairing to be considered valid (equivalent to the three processes visible in Figure 4). The initial rating opening determines whether a pairing is immediately valid when a player joins the queue; if two players in the queue have a rating difference smaller than the initial rating opening, they are immediately paired. The rating expansion determines how quickly more unbalanced pairings become acceptable by expanding the rating search range by that number each round; setting this value high makes players wait less but gives more unbalanced matches, while setting it low does the opposite. Finally, the required range overlap for a pairing determines whether a single player waiting can force an unbalanced pairing or if both players must be waiting for an unbalanced match to be valid. Two policies were tested, one where the ranges only had to overlap for the pairing to be valid called “just overlap” (meaning one player waiting a long time can force a player who just joined the queue into an unbalanced match), and one where the ranges must overlap by at least half each called “half overlap” (i.e., requiring each player’s range to overlap with the rating of their opponent, making it harder for unbalanced matches to be formed, but also making players wait significantly longer).

#### FINAL VARIABLES:

To summarise the chapter, the final variables are presented here in list format and are what the simulations use.

- Constant:
  - Pairwise Probability Comparator: Bradley-Terry Model
  - Number of Players: 1000
  - Skill Generation: Logarithmic Gaussian Curve
  - Simulations Run: 15
- Independent:
  - Rating systems tested: Elo, Glicko, Glicko-2, OpenSkill
  - Total Number of Matches: 5,000 & 50,000
  - Player-Joining Behaviour:
    - Single-round: All players available (players may play multiple matches in a single round, unlike other player-joining behaviours)
    - Multi-round: All players selected & 0.5 chance of selection
    - Continuous: 0.001 chance of selection
  - Matchmaking Policy:
    - Single-round: Random
    - Multi-round: Stairs, Batched & Skill Weighted Batched
    - Continuous: Box:
      - Initial Rating Opening: Various Tested
      - Range Expansion: Various Tested
      - Required Overlap: “Just overlap” & “Half overlap”

- Metrics:
  - Primary: Match Skill Disparity and Average Wait Time
  - Secondary: Rating Error

## V. RESULTS

### RATING ERROR:

The first simulations were run with random matchmaking. It is immediately confirmed by the rating errors obtained that Elo performs significantly worse than both Glicko (including Glicko-2) and OpenSkill. It also, expectedly, performs incredibly poorly with newer players (fewer matches); simply counting the number of wins is a better match result predictor with 10 matches per player (the results for this are available in the Appendix). Elo does perform better with 100 matches per player, but all three of the other systems perform significantly better than Elo.

Multi-round matchmaking was tested next. Ten-round and one hundred-round simulations were run, with all players available each round, for a total of 5,000 and 50,000 matches respectively, as well as a player availability of 100% and 50% each round (doubling the number of rounds for the simulations with a 50% selection probability). Figure 8 presents those results, along with the random matchmaking.

| Rating Error for 15 Simulations       | Elo     | Glicko  | Glicko2 | OpenSkill |
|---------------------------------------|---------|---------|---------|-----------|
| 5000 Random Matches                   | 0.08371 | 0.03854 | 0.03852 | 0.04159   |
| 10 Random Rounds                      | 0.08289 | 0.03839 | 0.03836 | 0.04140   |
| 10 Stairs Rounds                      | 0.10383 | 0.02714 | 0.02699 | 0.04487   |
| 10 Batched Rounds                     | 0.10002 | 0.02723 | 0.02773 | 0.03946   |
| 10 Skill Weighted Batched Rounds      | 0.10291 | 0.02729 | 0.02785 | 0.04193   |
| 20 Stairs Matchmaking Rounds          | 0.10112 | 0.02786 | 0.02802 | 0.04130   |
| 20 Batched Matchmaking Rounds         | 0.09914 | 0.02908 | 0.02906 | 0.03955   |
| 20 Skill Adjusted Matchmaking Rounds  | 0.10079 | 0.02845 | 0.02878 | 0.03994   |
| 50000 Random Matches                  | 0.01067 | 0.00785 | 0.00517 | 0.00756   |
| 100 Random Rounds                     | 0.01077 | 0.00794 | 0.00521 | 0.00751   |
| 100 Stairs Rounds                     | 0.04662 | 0.00726 | 0.00659 | 0.00390   |
| 100 Batched Rounds                    | 0.03117 | 0.00727 | 0.00640 | 0.00394   |
| 100 Skill Weighted Batched Rounds     | 0.03282 | 0.00730 | 0.00659 | 0.00390   |
| 200 Stairs Matchmaking Rounds         | 0.03697 | 0.00713 | 0.00659 | 0.00385   |
| 200 Batched Matchmaking Rounds        | 0.02652 | 0.00704 | 0.00600 | 0.00418   |
| 200 Skill Adjusted Matchmaking Rounds | 0.02980 | 0.00719 | 0.00630 | 0.00404   |

Figure 8: The rating error of different rating systems for different matchmaking policies from the simulations for Single-Round (light orange), Multi-Round (light blue), and Multi-Round with 50% of players available each round (light green)

Elo seems to be particularly disadvantaged by pairing players directly next to each other compared to randomly choosing a close player (especially with 100 matches per player). This can be explained by the fact that, because the Elo rating changes are capped by the  $k$  factor, two players (or a small group of players) who happen to have a similar skill and rating are (all) paired with each other every round; they stay close to each other in ratings because Elo is zero-sum and they have a similar skill level (therefore requiring a streak of wins or losses to break the loop, which is least likely with similarly skilled players), but have drastically incorrect ratings compared to other pairs (or small groups) of players (who may themselves be in a loop of being paired with each other). Allowing Elo to choose from more players makes it significantly easier to break out of that loop. This hypothesis is supported by the match skill disparity metric, which is discussed after.

Similarly, OpenSkill has a higher error with the “Stairs” matchmaking policy compared to the other two policies, but only for 5,000 matches. This difference is substantially smaller than the one Elo had, and is harder to identify the cause of. A possible reason is how OpenSkill’s  $\sigma$  (similar to rating deviation for Glicko) is treated for players with a low number of matches. The spread of  $\sigma$  between players in a simulation is quite low (only 2 percentage points over the initial value of 8.333, compared to Glicko’s 5 percentage points over its initial value of 350); this could cause OpenSkill to have a similar issue to Elo as the rating changes after a match are close to zero-sum. This is consistent with Elo’s similar small rating error increase in the ten-round simulation for the stairs matchmaking policy. This would also explain the error increase in the skill weighted batched matchmaking policy for both Elo and OpenSkill, as the problem would be similar to the stairs policy but more subtle.

The rating error calculated is fairly similar within a given rating system, regardless of the matchmaking policy chosen out of the latter three, except for those two exceptions above. This suggests that a rating system generally does not gain much accuracy from the exact specific variables chosen in the matchmaking algorithm chosen (such as the number of players selected in the batch matchmaking), but much more in the general way players are selected in the first place. The proximity of results for rating errors within each of Glicko, Glicko-2, and OpenSkill within 5,000 and 50,000 simulation runs are particularly indicative of this.

Perhaps most interesting within a single rating system is that having a matchmaking policy in place at all seems to increase the accuracy of Glicko and Glicko-2 for 5,000 matches, but makes it worse for 50,000 matches. OpenSkill does the opposite, preferring random matches when there are few of them, and preferring matchmaking with many matches. The likely reasoning for the Glicko systems is that they gain more information from matches which they predict are close at first (but may not actually be), while getting more varied information (with more matches) is better in the long run for its accuracy. OpenSkill is likely overfitting on the Bradley-Terry model when it has 50,000 matches; with 100 matches per player, the information it receives comes exactly from the model OpenSkill expects, and it can maximise the information gain from each match in the long run (while preferring player space exploration with fewer matches). This incredible accuracy would likely not hold in real circumstances. Indeed, with some more experiments (the results of which are presented in the Appendix), introducing even a slight arbitrary bias to the Bradley-Terry model reduces the gap between random matching and the matchmaking policies’ rating errors to less than 20% (compared to the initial simulations’ close to 100%).

Glicko and Glicko-2 have comparable rating errors, especially for 5,000 matches. However, at 50,000 matches, Glicko-2 has slightly more accuracy. The improvements made to Glicko-2 likely maximise its information gain on predictable models (such as the Bradley-Terry model). Reducing the starting volatility of Glicko-2 also significantly reduces its rating error in simulations; because there is no skill volatility (all player skills are fixed and predictable because of the model), a high volatility delays and reduces the information gains that can be obtained from the matches in such a predictable environment. Real-world match data would likely not benefit from reducing volatility.

The Glicko-2 volatility has, in fact, become a point of contention in many online games due to the possibility of artificially increasing one’s volatility by purposefully losing several (hundreds of) matches against players Glicko-2 would have expected the player to win against (and winning against players the system expected the player to lose against). While in the long term the player’s rating will still converge to their true skill, a good win streak could cause the player’s rating to inflate much higher than that same player would have with a low volatility. “Farming” volatility in such a way is very difficult and time-consuming for a player however, especially with only one match per Glicko rating period.

Nevertheless, making a small change to Glicko-2 to protect volatility from changing both significantly with one match and significantly over time would be beneficial.

Glicko (or Glicko-2) and OpenSkill have comparable rating errors. Interestingly, Glicko performs better with fewer matches than more compared to OpenSkill, with OpenSkill being better than Glicko in the long run. The assumptions they make on player populations and their skills are likely different, and how they use the information from the matches differs depending on how likely the systems (or, in fact, their designers) think real-player populations differ from the pairwise probability model (e.g., the Bradley-Terry model). Glicko uses more of the information given for a player's first few matches compared to OpenSkill and, as discussed above, OpenSkill matches the data more closely to the Bradley-Terry model once it has more data to base itself on.

Therefore, according to the rating error, the preferred rating systems are Glicko-2 and OpenSkill. Glicko, being slightly less complicated to implement, is also an acceptable alternative. Elo, however, should be avoided, if possible, as it struggles to reach prediction capabilities even close to other modern rating systems. The rating error makes no definitive suggestions as to which matchmaking policy to prefer for accuracy, the match skill disparity will be a more determining factor.

The choice of matchmaking policy does seem to affect how rating systems perform in the most drastic cases (namely random pairings compared to actual matchmaking). While random matchmaking is useful at the beginning to gather data about how players perform, more information is gained from who wins close matches compared to a player already being predicted to win and winning (which is much more likely to happen with random pairings). The information a match could have provided is lost if the winner is already expected by the rating system. This leads to the natural fact that lopsided matches should be avoided. This is already the case because of the objective of Skill-Based Matchmaking, but rating systems also benefit from this.

#### NON-CONTINUOUS MATCH SKILL DISPARITY:

The choice for which rating system performs best with match skill disparity is, perhaps surprisingly, not a contest. Glicko-2 performs better by far, with even Glicko outperforming OpenSkill regardless of the chosen matchmaking policy. Glicko-2 always has a lower match skill disparity than Glicko, which itself always has a lower match skill disparity than OpenSkill. While OpenSkill may be a better skill predictor given enough time, Glicko manages to more quickly estimate a player's skill and, therefore, always give them as close a match as possible. Both Glicko and OpenSkill are still very reasonable alternatives; OpenSkill is especially useful in games involving more than two players as it offers many options for those kinds of games, without the need to modify Elo or Glicko. Elo also offers acceptable results, but does not achieve the same level of balance as the other systems do, especially over longer periods of time.

Random matchmaking gives a baseline match skill disparity to compare the matchmaking policies to. "Batched" matchmaking performs rather poorly compared to the other policies. "Skill Weighted Batched" performs better, but the best is by far the "Stairs" matchmaking policy; even though the rating error was sometimes higher for Elo and OpenSkill, two players who are close in skill are also more likely to be close in rating (even if their rating is incorrect compared to other players). This suggests that, for continuous matchmaking, there are no advantages to purposefully choosing a player different than the one currently closest in skill in the queue.

When running twice as many rounds, with each individual player having a 50% chance of being available each round (therefore maintaining the total number of matches the same), the match skill disparities maintain a similar pattern between the three matchmaking policies. While reducing the

| Matchmaking Error for 15 Simulations  | Elo     | Glicko  | Glicko2 | OpenSkill |
|---------------------------------------|---------|---------|---------|-----------|
| 5000 Random Matches                   | 0.62567 | 0.62567 | 0.62567 | 0.62567   |
| 10 Random Rounds                      | 0.62300 | 0.62300 | 0.62300 | 0.62300   |
| 10 Stairs Rounds                      | 0.42952 | 0.42934 | 0.42810 | 0.42985   |
| 10 Batched Rounds                     | 0.46198 | 0.44220 | 0.44289 | 0.44930   |
| 10 Skill Weighted Batched Rounds      | 0.43420 | 0.42936 | 0.42992 | 0.43259   |
| 20 Stairs Matchmaking Rounds          | 0.44605 | 0.42506 | 0.42592 | 0.43836   |
| 20 Batched Matchmaking Rounds         | 0.47902 | 0.45668 | 0.45639 | 0.47009   |
| 20 Skill Adjusted Matchmaking Rounds  | 0.45795 | 0.43731 | 0.44001 | 0.45071   |
| 50000 Random Matches                  | 0.62833 | 0.62833 | 0.62833 | 0.62833   |
| 100 Random Rounds                     | 0.62661 | 0.62661 | 0.62661 | 0.62661   |
| 100 Stairs Rounds                     | 0.18864 | 0.13682 | 0.11143 | 0.14552   |
| 100 Batched Rounds                    | 0.24510 | 0.19011 | 0.17005 | 0.19653   |
| 100 Skill Weighted Batched Rounds     | 0.21961 | 0.16285 | 0.14049 | 0.16975   |
| 200 Stairs Matchmaking Rounds         | 0.19800 | 0.13632 | 0.11151 | 0.14445   |
| 200 Batched Matchmaking Rounds        | 0.30105 | 0.26202 | 0.24965 | 0.26728   |
| 200 Skill Adjusted Matchmaking Rounds | 0.25126 | 0.20188 | 0.18327 | 0.20830   |

Figure 9: The matchmaking error for the rating systems for 5,000 or 50,000 matches. Single-round random matchmaking policy in light orange, multi-round matchmaking in light blue, and multi-round matchmaking with 50% of players each round in light green.

number of players available each round increases match skill disparity because the pairings may be less optimal, increasing the number of rounds allows the system to tell players apart more, reducing match skill disparity. The reduction of available players generally seems to prevail, with match skill disparity increasing in most cases compared to their 10/100 round comparisons. However, the “Stairs” policy with Glicko, Glicko-2 with 5,000 matches, and OpenSkill with 50,000 matches benefit more from the additional number of rounds. The increase in the number of rounds was more beneficial than the decrease in the number of players each round was detrimental. Still, this error decrease is very small and does not affect the choice of matchmaking policy.

Given the results from single-round and multi-round simulations, the match skill disparity errors suggest that the choice of a matchmaking policy is independent from that of the rating system. Choosing a rating system does not affect how well a matchmaking policy performs compared to other matchmaking policies. This is incredibly useful as it allows the separation of the development and choice of the rating system and policy. They can each individually be fine-tuned for the specific game.

#### CONTINUOUS MATCHMAKING:

Because hyper-optimizing match skill disparity by disregarding wait time or vice-versa is not useful, both metrics need to be considered with one another’s context. Accepting every pairing will reduce the wait time mean square error to around 0.5, but is close to equivalent to random matchmaking, and making a player wait for the perfectly balanced match is also not feasible. These two targets are diametrically opposed, improving one naturally hinders the other. This leads to a set of pareto efficient matchmaking policies, rather than a single universal matchmaking policy (Pareto optimal values or sets of values are achieved when improving one characteristic further would necessarily lead to the deterioration of another). These pareto optimal values may also be different depending on the game and how and how often players join the matchmaking queue.

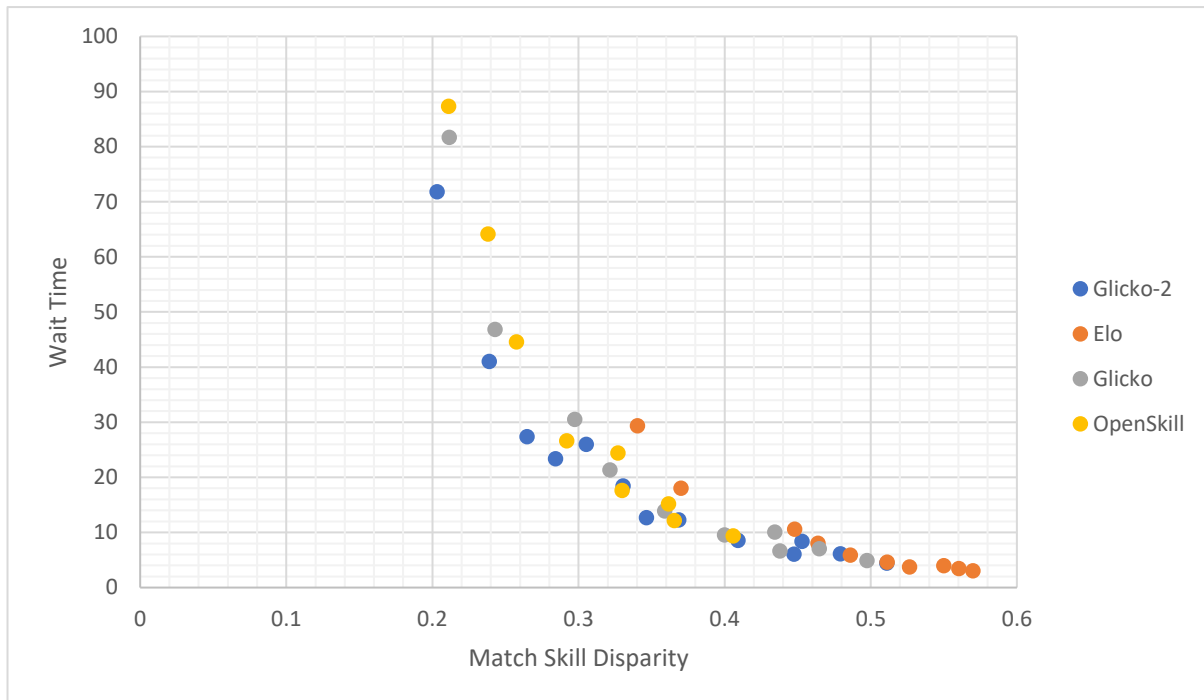


Figure 10: Wait Time vs Match Skill Disparity for various "just overlap" continuous matchmaking policies.

Figure 10 presents the wait time and match skill disparity errors from different "just overlap" policies. The full results for continuous matchmaking of "just overlap" policies, with the two metrics (match skill disparity and average wait time), are presented in Figure 11, Figure 12, and Figure 13. It is important to note that the rating systems are not directly comparable there, even if supposedly using the same matchmaking policy, as the rating scales are different between each system (OpenSkill has a unique scale as it uses a different order of magnitude for its ratings); instead, Figure 10 should be used. Figure 11 presents both metrics summed, with the wait time mean square error arbitrarily divided by 100. Other ways of summing the two errors would also be appropriate, depending on the specific use case.

Reducing the wait time as much as possible approaches the match skill disparity of random matchmaking. Conversely, the match skill disparity seems very difficult to reduce below 0.2 without making players wait for hundreds of rounds. The "half overlap" policies manage to go below 0.2, but the wait time error surges to 500 for Glicko and 2000 for OpenSkill, with hundreds of players having to wait more than 100 rounds for their match.

Even within a single rating system, pareto improvements can still be made. Glicko-2, with a rating opening of 50 and rating expansion of 24 performs better on both wait time and match skill disparity compared to 100 and 12 respectively. These kinds of improvements are important as they are an easy way to improve a game's matchmaking without resorting to more complicated matchmaking policies.

The "half overlap" policies are all pareto regressions compared to "just overlap" policies (except perhaps at the extremities, but ignoring one metric for the other is not useful for a matchmaking policy), always being worse on both match skill disparity and wait time. "Half overlap" policies do have one advantage which is that they will only pair a player who just joined the queue with other players close to them in rating; "just overlap" may pair that player with another who has waited a long time even if the match is unbalanced. This can make the player who joined feel like the matchmaking policy treated them unfairly, prioritising the need of the waiting player rather than theirs.

| Initial |           | Expansion |           | Rating Systems |         |          |           |
|---------|-----------|-----------|-----------|----------------|---------|----------|-----------|
| Others  | OpenSkill | Others    | OpenSkill | Elo            | Glicko  | Glicko-2 | OpenSkill |
| 50      | 1         | 6         | 0.125     | 0.63401        | 1.02838 | 0.92132  | 1.77956   |
| 50      | 1         | 12        | 0.25      | 0.55045        | 0.71096 | 0.64919  | 1.08444   |
| 100     | 2         | 12        | 0.25      | 0.55397        | 0.60235 | 0.56498  | 0.87921   |
| 100     | 2         | 20        | 0.5       | 0.54431        | 0.53459 | 0.51456  | 0.70306   |
| 100     | 2         | 36        | 0.75      | 0.54493        | 0.49784 | 0.49130  | 0.55799   |
| 100     | 2         | 60        | 1.25      | 0.55711        | 0.49539 | 0.49446  | 0.50610   |
| 100     | 2         | 100       | 2         | 0.56363        | 0.50428 | 0.50783  | 0.48712   |
| 200     | 4         | 24        | 0.5       | 0.58964        | 0.53496 | 0.53668  | 0.57093   |
| 200     | 4         | 48        | 1         | 0.59482        | 0.53513 | 0.54046  | 0.51306   |
| 200     | 4         | 96        | 2         | 0.60042        | 0.54650 | 0.55531  | 0.49926   |

Figure 11: Results of Continuous Matchmaking for a pairing policy of "just overlap", for different values of the initial rating and rating expansion. The mean square error of wait time is divided by 100 and added to the match skill disparity.

| Initial |           | Expansion |           | Match Skill Disparity of Rating Systems |         |          |           |
|---------|-----------|-----------|-----------|-----------------------------------------|---------|----------|-----------|
| Others  | OpenSkill | Others    | OpenSkill | Elo                                     | Glicko  | Glicko-2 | OpenSkill |
| 50      | 1         | 6         | 0.125     | 0.34025                                 | 0.21135 | 0.20315  | 0.18788   |
| 50      | 1         | 12        | 0.25      | 0.37008                                 | 0.24285 | 0.23887  | 0.21104   |
| 100     | 2         | 12        | 0.25      | 0.44786                                 | 0.29733 | 0.30518  | 0.23806   |
| 100     | 2         | 20        | 0.5       | 0.46376                                 | 0.32145 | 0.33047  | 0.25751   |
| 100     | 2         | 36        | 0.75      | 0.48585                                 | 0.35884 | 0.36856  | 0.29192   |
| 100     | 2         | 60        | 1.25      | 0.51108                                 | 0.39980 | 0.40909  | 0.32969   |
| 100     | 2         | 100       | 2         | 0.52651                                 | 0.43777 | 0.44740  | 0.36544   |
| 200     | 4         | 24        | 0.5       | 0.54992                                 | 0.43429 | 0.45276  | 0.32688   |
| 200     | 4         | 48        | 1         | 0.56029                                 | 0.46479 | 0.47919  | 0.36149   |
| 200     | 4         | 96        | 2         | 0.56991                                 | 0.49733 | 0.51100  | 0.40583   |

Figure 12: The match skill disparity of different continuous matchmaking policies. All results here use the "just overlap"

| Initial |           | Expansion |           | Wait Time MSE |          |          |           |
|---------|-----------|-----------|-----------|---------------|----------|----------|-----------|
| Others  | OpenSkill | Others    | OpenSkill | Elo           | Glicko   | Glicko-2 | OpenSkill |
| 50      | 1         | 6         | 0.125     | 29.37616      | 81.70257 | 71.81772 | 159.16766 |
| 50      | 1         | 12        | 0.25      | 18.03746      | 46.81154 | 41.03207 | 87.34036  |
| 100     | 2         | 12        | 0.25      | 10.61107      | 30.50231 | 25.97966 | 64.11535  |
| 100     | 2         | 20        | 0.5       | 8.05525       | 21.31453 | 18.40873 | 44.55551  |
| 100     | 2         | 36        | 0.75      | 5.90791       | 13.89947 | 12.27369 | 26.60721  |
| 100     | 2         | 60        | 1.25      | 4.60356       | 9.55886  | 8.53783  | 17.64156  |
| 100     | 2         | 100       | 2         | 3.71193       | 6.65041  | 6.04279  | 12.16820  |
| 200     | 4         | 24        | 0.5       | 3.97232       | 10.06755 | 8.39278  | 24.40519  |
| 200     | 4         | 48        | 1         | 3.45277       | 7.03386  | 6.12672  | 15.15671  |
| 200     | 4         | 96        | 2         | 3.05114       | 4.91657  | 4.43030  | 9.34355   |

Figure 13: The wait time mean square error of different policies for continuous matchmaking for the "just overlap" policy.

Very high or low rated players are often waiting the most, because there are less players at their level. In real matchmaking queues, particularly low-rated players (almost always new players) are often rare because they naturally improve as they play more matches; the rating system generally does not have the time to give them a precise low rating because they will have improved before then (the standard deviation of lower-rated players is often higher than those of other players). Similarly, in real matchmaking queues, high-rated players often play much more than other players, and therefore find each other more easily. This forms a limitation of the simulation which is difficult to fix in a non-arbitrary manner as the wait time error may be higher than it would otherwise be because of those differences in player behaviour. Using real-world data could fix this issue, but it would itself always be biased by the matchmaking policy used by the game where that data originates from. Some players who join the queue also leave after some time if no match is found, which is also very dependent on the specific game, the expectations of the player, and more. Overall, developers should optimise their matchmaking policy based on their specific game, community, and player behaviours.

#### AN EXAMPLE WITH *TETR.IO*:

Developers can optimize their matchmaking policies using simulations. To show this, *TETR.IO* will be used as an example, with arbitrary values for illustrative purposes, roughly based on *TETR.IO*'s player count. Simulated here is the first day after a "season reset", where all players' ratings are reset. 30,000 rounds are run (for one matchmaking round every three seconds), with 2,000 players (with a selection probability of 0.0005 to keep, on average, one player joining every round). Additionally, a player cannot join the queue for ten minutes after they've been matched (simulating them playing the match) but, once they do finish, they have a 60% chance to join the queue again (a player who's just finished playing a match is much more likely to join the queue again compared to a random player). With these numbers, around 30,000 matches are played during the first day, for an average of 15 matches per player (some players have more than 100 matches, with some others having less than 5).

Because of the evidence for rating system and matchmaking policy independence for matchmaking performance, and since *TETR.IO* uses Glicko-2, the rating system chosen will be Glicko-2 in order to find more optimal matchmaking policies. Just like with the previous continuous matchmaking experiments, different combinations of initial opening and rating expansion are tested.

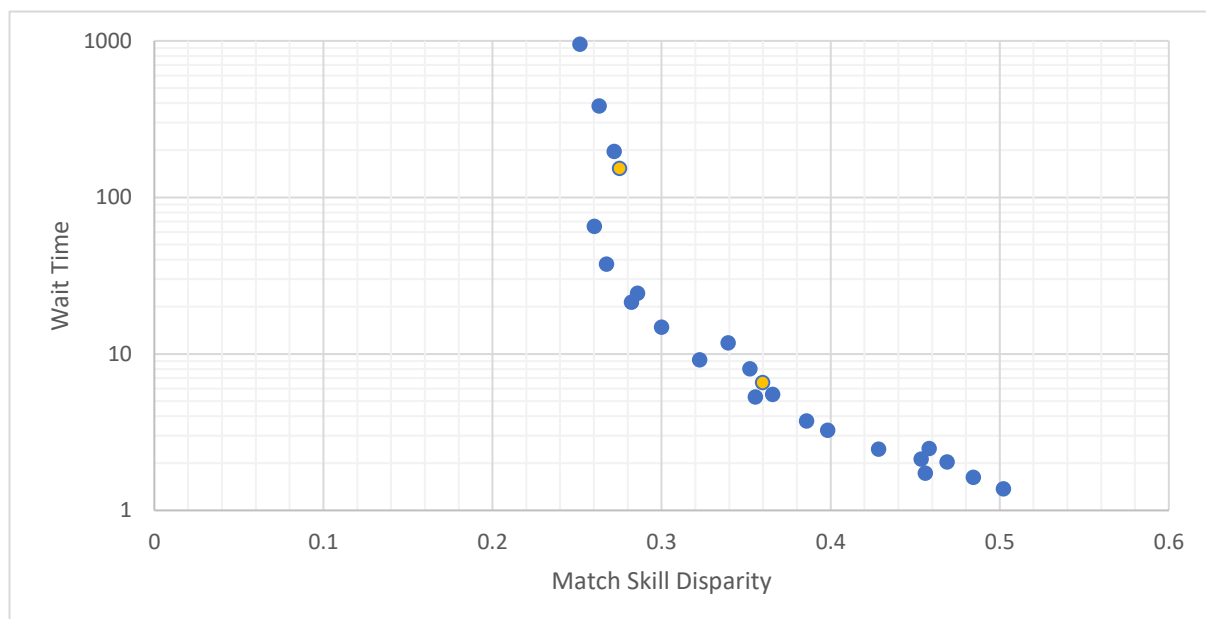


Figure 14: A scatter plot for the *TETR.IO* example of wait time and match skill disparity. The two orange values use dynamic rating openings (based on the player's rating) rather than a constant value.

As seen in Figure 14, and similar to the continuous matchmaking results, several policies are pareto regressions compared to others as both their wait time and match skill disparity are worse (the full results are available in the Appendix). Some of the pareto optimal matchmaking policies include those with an initial rating opening of 25 (which is a particularly low initial rating opening), and generally prefer having a large rating expansion, as it sacrifices only a small amount of match skill disparity for a relatively larger decrease in wait time.

Two matchmaking policies, represented by orange dots in Figure 14, use dynamic rating openings rather than a constant value. These are based on Lichess' own matchmaking policy (which also uses Glicko-2) (Lichess, 2025); one uses the "half overlap" policy and the other "just overlap". The initial rating opening are different based on the player's rating, with especially high ratings starting with a higher initial rating opening. However, as seen in the scatter plot, they are both pareto regressions compared to other policies. This does not mean that dynamic rating openings are necessarily always worse, there may be other values for which it would be more performant to use dynamic rating openings, and this used Lichess' rating openings on rough *TETR.IO* values, which could also bias the result against dynamic rating openings.

In order to choose one matchmaking policy, both wait time and match skill disparity need to be comparable numerically, similar to what was done in Figure 11. This can be done by dividing the wait time by 500 and summing both; this would result in the optimal matchmaking policy of 25 for the initial rating opening and 12 for the rating expansion. But punishing wait time more severely, such as by dividing it by 100 instead of 500, gives an optimal matchmaking policy of 50 for the opening and 50 for the expansion. There are, just as with other sets of pareto optimal values, multiple optimal matchmaking policies depending on the weighting given to each metric. For *TETR.IO*, where skill disparity is arguably more important (because a significantly weaker player stands no chance of winning against a stronger player) and having players wait slightly longer is acceptable to avoid this, a 25 and 12 policy is perhaps most appropriate.

While more complex matchmaking policies were not tested (such as those with dynamic rating expansions, but also more comprehensive testing of dynamic rating opening), it is easy to choose a matchmaking policy which is a pareto regression compared to another policy without running simulations. More extensive testing could also be performed using optimization tools such as gradient descent or machine learning. Nonetheless, these simulations provide a basic framework which can be used as preliminary testing for matchmaking policy testing before implementation in games.

## VI. CONCLUSION

There have been no results which contradict the independence between the choice of rating system and the performance of matchmaking policies compared to each other, supporting the theoretical result shown by Nozawa (2025). This independence between the rating system and matchmaking policy allows developers to work on the rating system separately from the matchmaking system, and ensures that improvements made to the rating system will not negatively impact the matchmaking policy.

Glicko-2 has shown greater accuracy and performance in matchmaking than the other rating systems, and is the recommended rating system for one-versus-one games provided that volatility is monitored to ensure players cannot maliciously influence it. Glicko and OpenSkill are acceptable alternatives that offer close results to that of Glicko-2. Elo should be avoided if possible as it offers significantly worse performance compared to the other rating systems.

The simulations suggest the matchmaking policy has no reason to choose an opponent other than the closest one in the matchmaking queue. This is especially true as the number of available opponents decreases. There are a few limitations to this result. If other factors are considered important for the game, such as latency, these may take priority if the currently most balanced match would be unfavourable to these other factors. Additionally, these results were run for one-versus-one matchmaking queues; while there are no immediate reasons as to why this would not hold for free-for-all or team games, future research would still be needed to verify this. Regardless, for one-versus-one games, if the other factors between the two players are the same, the matchmaking policy should always pick the closest available player in terms of rating, or defer the pairing to a future round and wait.

Most games that use the “box” matchmaking policy, or an equivalent policy, are generally “half overlap” policies (requiring both players to wait a long time before a more unbalanced pairing is considered acceptable). However, as shown by the continuous matchmaking results, this may not be optimal, and a “just overlap” policy (one where only one player needs to wait significantly) may be better suited to find more balanced matches, more quickly. It is possible that the specific metrics chosen, or the way the simulations were run, made it specifically biased towards “just overlap” policies. The matchmaking tested was on a specific player-joining behaviour which does not mirror real player behaviour. Confirming whether “just overlap” policies are more performant requires more research, and likely necessitates A/B testing on a live online competitive game. Developers implementing a “box” matchmaking policy are encouraged to consider “just overlap” policies as it may be beneficial to reduce wait times and obtain more balanced pairings.

These simulations can also provide a basic framework to test matchmaking policies against, given specific player-joining behaviours (taken from data in a game for example). This can be used to find more optimal matchmaking policies compared to the one currently in use in the game, and/or to choose a matchmaking policy which is pareto optimal.

Extensive research is still needed in both rating systems and matchmaking policies. Rating systems, while very performant in specific circumstances, still struggle in more general scenarios without modification, and there exists no universal framework to easily modify them for a specific use case. More volatile games (such as those where luck may take a more significant part in the result and matches may last only a few minutes) are common, but do not easily work with rating systems directly. The Bradley-Terry model is very predictable, but does not mirror real-player behaviour. More general frameworks for modifying and optimizing matchmaking policies are also lacking, with most research

focusing on a single specific use case. Still, the positive result for independence of rating systems and matchmaking policies when considering the matchmaking performance means developers and researchers can largely focus on developing more performant rating systems and more optimized matchmaking policies, separately.

## REFERENCES

- Biasutti, M. (2011). Flow and Optimal Experience. (M. A. Runco, & S. R. Pritzker, Eds.) *Encyclopedia of Creativity (Second Edition)*, 522-528. doi:10.1016/B978-0-12-375038-9.00099-6
- Blavatsky, P. R. (2024). A behavioral definition of loss aversion. *Economics Letters*, 235(111555). doi:10.1016/j.econlet.2024.111555
- Chen, Z., Xue, S., Kolen, J., Aghdaie, N., Zaman, K. A., Sun, Y., & Seif El-Nasr, M. (2017). EOMM: An Engagement Optimized Matchmaking Framework. *26th International Conference on the World Wide Web* (pp. 1143-1150). Perth, Australia: International World Wide Web Conference Committee. doi:http://dx.doi.org/10.1145/3038912.3052559
- Chess.com. (n.d.). *Elo Rating System - Chess Terms*. Retrieved July 2026, from Chess.com: <https://www.chess.com/terms/elo-rating-chess>
- Csikszentmihalyi, M. (1990, January). Flow: The Psychology of Optimal Experience. *Harper & Row*. Retrieved from [https://www.researchgate.net/publication/224927532\\_Flow\\_The\\_Psychology\\_of\\_Optimal\\_Experience](https://www.researchgate.net/publication/224927532_Flow_The_Psychology_of_Optimal_Experience)
- Ebtekar, A., & Liu, P. (2021). Elo-MMR: A Rating System for Massive Multiplayer Competitions. *Web Conference 2021*. Ljubljana, Slovenia: International World Wide Web Conference Committee. doi:https://doi.org/10.1145/3442381.3450091
- Elo, A. E. (1961, June). The USCF Rating System. *Chess Life*, 160-161. Retrieved July 2026, from [https://uscf1-nyc1.aodhosting.com/CL-AND-CR-ALL/CL-ALL/1961/1961\\_06.pdf](https://uscf1-nyc1.aodhosting.com/CL-AND-CR-ALL/CL-ALL/1961/1961_06.pdf)
- Evidently AI Team. (2025, February 14). *Normalized Discounted Cumulative Gain (NDCG) explained*. Retrieved August 2026, from Evidently AI: <https://www.evidentlyai.com/ranking-metrics/ndcg-metric>
- FIDE. (2025, August 25). *Anniversary of Arpad Elo – rating system that changed chess world*. Retrieved July 2026, from FIDE: <https://www.fide.com/anniversary-of-arpad-elo-rating-system-that-changed-chess-world/>
- FIDE. (2026, August 06). *Download latest official FIDE Rating list*. Retrieved August 2026, from FIDE Ratings: [https://ratings.fide.com/download\\_lists.phtml](https://ratings.fide.com/download_lists.phtml)
- Glickman, M. E. (1999). Parameter estimation in large dynamic paired comparison experiments. *Applied Statistics*, 48, 377-394. Retrieved August 2026, from <https://www.glicko.net/research/glicko.pdf>
- Glickman, M. E. (2001). Dynamic paired comparison models with stochastic variances. *Applied Statistics*, 28(6), 673-689. doi:10.1080/02664760120059219
- Glickman, M. E. (2001). *Example of the Glicko-2 system*. Boston University, Boston. Retrieved August 2026, from <https://www.glicko.net/glicko/glicko2.pdf>
- Glickman, M. E. (n.d.). *acjpaper*. Retrieved July 2026, from Glicko: <https://www.glicko.net/research/acjpaper.pdf>
- Glickman, M. E. (n.d.). *The Glicko system*. Harvard University, Boston. Retrieved August 2026, from <https://www.glicko.net/glicko/glicko.pdf>

- Herbrich, R., Minka, T., & Graepel, T. (2007). *TrueSkill™: A Bayesian Skill Rating System*. Retrieved July 2026, from Microsoft Research: [https://www.microsoft.com/en-us/research/wp-content/uploads/2007/01/NIPS2006\\_0688.pdf](https://www.microsoft.com/en-us/research/wp-content/uploads/2007/01/NIPS2006_0688.pdf)
- King, D. L., Russell, A. M., Delfabbro, P. H., & Polisena, D. (2020). Fortnite microtransaction spending was associated with peers' purchasing behaviors but not gaming disorder symptoms. *Addictive Behaviors*, 104. doi:10.1016/j.addbeh.2020.106311
- Lichess. (2025). *lila/modules/pool/src/main/MatchMaking.scala at master | lichess.org/lila*. Retrieved August 2026, from Github: <https://github.com/lichess-org/lila/blob/master/modules/pool/src/main/MatchMaking.scala>
- Lichess. (2026, August). *Weekly Blitz Rating Distribution*. Retrieved August 2026, from Lichess: <https://lichess.org/stat/rating/distribution/blitz>
- Nozawa, W. (2025, December 26). *Mean-Field Analysis and Optimal Control of a Dynamic Rating and Matchmaking System*. Retrieved August 2026, from arXiv: <https://arxiv.org/pdf/2512.22038>
- Shev, A., Fujii, K., Hsieh, F., & McCowan, B. (2014, December). Systemic Testing on Bradley-Terry Model against Nonlinear Ranking Hierarchy. *PLOS ONE*, 9, 1-18. doi:10.1371/journal.pone.0115367
- Statista Research Department. (2025, July 31). *Concurrent user record in top video games 2025*. Retrieved August 2026, from Statista: <https://www.statista.com/statistics/1620202/concurrent-user-record-for-select-video-games/>
- Valve. (2026, August 2). *Most Played*. Retrieved August 2, 2026, from Steam: <https://store.steampowered.com/charts/mostplayed>
- Volk-Jesussek, H. (2026, January 26). *Spearman's rank correlation coefficient*. Retrieved August 2026, from numiqo: <https://numiqo.com/tutorial/spearman-correlation>
- Weng, R. C., & Lin, C.-J. (2011). A Bayesian Approximation Method for Online Ranking. *Journal of Machine Learning Research*, 12(9), 267-300. Retrieved August 2026, from <http://jmlr.org/papers/v12/weng11a.html>
- World Tennis. (2019). *atp-and-wta-points*. Retrieved August 2026, from ITF Tennis: <https://www.itftennis.com/media/2213/atp-and-wta-points.pdf>
- Wu, W., Niezink, N., & Junker, B. (2022, November 5). A Diagnostic Framework for the Bradley–Terry Model. *Journal of the Royal Statistical Society Series A: Statistics in Society*, 185(Supplement\_2), S461-S484. doi:10.1111/rssa.12959
- Zhang, C., Wang, K., Chen, H., Fan, G., Li, Y., Wu, L., & Zheng, B. (2022). QuickSkill: Novice Skill Estimation in Online Multiplayer Games. *31st ACM International Conference on Information & Knowledge Management* (pp. 3644-3653). Atlanta, USA: Association for Computing Machinery. doi:<https://doi.org/10.1145/3511808.35570701>

## APPENDIX: ADDITIONAL EXPERIMENT RESULTS

A few more experiments were performed to confirm some hypotheses, the results of which are available here.

### ELO VS. WIN COUNT:

The rating error for Elo for 5000 random matches was 0.08371. Three more rating systems were developed to compare against it. “Random”, which simply predicts a probability of 0.5 for every player and match; “Wins” where the rating of the player is equal to the number of match wins they have, and the probability of winning is given by the player’s rating divided by the sum of both player’s ratings (similar to the Bradley Terry model), giving 0.5 if the sum equals 0; and “Difference”, which modifies the player’s rating by -1 for a loss, and 1 for a win, then the difference between both player’s rating, divided by 10, gives the probability of that player winning (if the difference between the ratings was greater than 10, the probability was capped at 1 or 0).

This gives the following table of rating errors, with the other rating systems also added as comparison.

| <i>Matches</i> | <i>Elo</i> | <i>Glicko</i> | <i>Glicko-2</i> | <i>OpenSkill</i> | <i>Random</i> | <i>Wins</i> | <i>Difference</i> |
|----------------|------------|---------------|-----------------|------------------|---------------|-------------|-------------------|
| <i>5,000</i>   | 0.08371    | 0.03854       | 0.03852         | 0.04159          | 0.15801       | 0.07820     | 0.06551           |
| <i>50,000</i>  | 0.01067    | 0.00785       | 0.00517         | 0.00756          | 0.15704       | 0.05123     | 0.04330           |

While Elo does perform better eventually (with 50,000 matches), counting the number of wins is a better way of predicting the winner of a match for 5,000 matches.

### BRADLEY-TERRY BIAS FOR OPENSkill:

By adding an arbitrary bias to the Bradley-Terry model (explicitly giving a 1.1 skill multiplication to the player with a lower index), OpenSkill’s rating error difference between random and stairs for 50,000 matches almost completely disappears (the difference now closer to 20% rather than 100%), which likely confirms that OpenSkill is hyper-optimizing on the Bradley-Terry model after enough matches.

The results for four different simulations runs with the Bradley-Terry arbitrary bias are present here in the table.

| <i>Matchmaking Policy</i>      | <i>Elo</i> | <i>Glicko</i> | <i>Glicko-2</i> | <i>OpenSkill</i> |
|--------------------------------|------------|---------------|-----------------|------------------|
| <i>5,000 matches random</i>    | 0.08918    | 0.04836       | 0.04828         | 0.04939          |
| <i>5,000 matches “Stairs”</i>  | 0.10385    | 0.03333       | 0.03364         | 0.04624          |
| <i>50,000 matches random</i>   | 0.01635    | 0.01472       | 0.01040         | 0.00823          |
| <i>50,000 matches “Stairs”</i> | 0.02384    | 0.00974       | 0.00817         | 0.00718          |

*TETR.IO* EXAMPLE:

The following table represents the results of the *TETR.IO* example from the end of the results chapter. The first number is the initial opening, and the second number the rating expansion each round. “Alt” signifies it used “half overlap” instead of “just overlap”, and Lichess was based on the dynamic initial openings used by Lichess (i.e., 130 for players with a rating below 1000, 100 for players with a rating between 1000 and 1500, and their rating divided by 15 otherwise).

| <i>Policy</i>   | Match<br>Skill<br>Disparity | Wait<br>Time |
|-----------------|-----------------------------|--------------|
| 25, 3           | 0.260249                    | 65.42988     |
| 25, 6           | 0.267276                    | 37.41132     |
| 25, 12          | 0.282147                    | 21.36849     |
| 50, 6           | 0.285804                    | 24.46556     |
| 50, 12          | 0.300015                    | 14.85033     |
| 50, 24          | 0.322444                    | 9.151513     |
| 50, 50          | 0.355479                    | 5.299669     |
| 50, 100         | 0.398096                    | 3.250975     |
| 50, 200         | 0.453368                    | 2.133304     |
| 100, 6          | 0.339219                    | 11.78911     |
| 100, 12         | 0.352181                    | 8.046773     |
| 100, 24         | 0.365572                    | 5.52336      |
| 100, 48         | 0.385603                    | 3.721908     |
| 100, 100        | 0.428308                    | 2.462298     |
| 100, 200        | 0.455984                    | 1.734199     |
| 200, 24         | 0.458151                    | 2.489373     |
| 200, 48         | 0.468832                    | 2.037168     |
| 200, 96         | 0.484324                    | 1.626383     |
| 200, 200        | 0.502106                    | 1.370757     |
| Alt 50, 6       | 0.251576                    | 953.4877     |
| Alt 50, 12      | 0.262984                    | 384.0399     |
| Alt 100, 12     | 0.271939                    | 197.1087     |
| Alt Lichess, 12 | 0.275232                    | 153.0842     |
| Lichess, 12     | 0.359779                    | 6.56039      |